DÉPARTEMENT
de PHYSIQUE

TP N° 2 : Matrices, tableaux et listes

Durée esitmée | 2 **séance de** $1h30$

I | But de la manipulation

Le but de cette manipulation est de manipuler les structures des données complexes avec le langage R?. Les thèmes traités sont les suivants :

1. les matrices ,
2. les tableaux de données ,
3. les listes.

Des exercices d'application son ensuite donnés à la fin de chaque paragraphe.

(C) Dr. ABBASSI A.
Department of Physic
| UM

II | Les matrices sous R

Les matrices, comme les vecteurs, sont de mode[1] quelconque, mais elles ne peuvent pas contenir des éléments de nature différente (c'est à dire qu'une matrice ne peut pas contenir des valeurs numériques et du texte par exemple). La syntaxe de base pour créer une matrice est : `matrix(vec, nrow = n,ncol = p,byrow = T)` où `vec` est le vecteur contenant les éléments de la matrice, qui seront **rangés en colonne** (sauf si l'option `byrow = T` est choisie).

Saisir le script suivant dans l'environnement de RStudio :

1. type

```
1
2  # Creation d'une matrice
3
4  vect_a <- 1 : 20 # vecteur
5  # matrice de dimension 5 * 4 a partir du vecteur vect_a
       (les valeurs dont rangees par colonne)
6  matrix_1 <- matrix(vect_a,  nrow = 5)
7  matrix_1
8  # matrice de dimension 5 * 4 a partir du vecteur vect_a
       (les valeurs dont rangees par ligne)
9  matrix_2 <- matrix(vect_a,  nrow = 5, byrow = TRUE)
10
11 # Operations sur les matrices
12 matrix_3 = t(matrix_2) # transposee de matrix_2 | 4 * 5
13 matrix_prod <- matrix_1 * matrix_2 # produit terme a
       terme
14 matrix_prod_1 <- matrix_3% * %matrix_2 # produit de
       matrices
15 log(matrix_1) ; sum(matrix_1)
16
17 # Extraction de sous matrices
18
19 dim(matrix_prod) #affiche les dimensions de matrix_prod_
       1
20 matrix_prod_1[3,2] #Extraction de l'element [3,2] de la
       matrice matrix_prod_1
21 matrix_prod_1[3,2] <- 5 #Change le coeffcient [3,2] de
```

```
    la matrice
22  matrix_prod_1[,2] #Extraction de la deuxieme colonne de
    matrix_prod_1
23  matrix_prod_1[c(3,4),] #Extraction des troisieme et
    quatrieme lignes de matrix_prod_1
24
25  matrix_prod_1[-2,] #Suppression de la deuxieme ligne de
    matrix_prod_1
26  matrix_prod_1[,-c(2,4)] #Suppression des deuxieme et
    quatrieme colonnes de matrix_prod_1
27  matrix_prod_1[,2] #Extraction de la deuxieme colonne de
    matrix_prod_1
28  matrix_prod_1[1,] > 600 #Test sur la premiere ligne de
    la matrice
29  index <- which(matrix_prod_1[1,] > 600)
30  matrix_prod_1[,index] #Extraction des colonnes de matrix
    _prod_1 telles que la premiere ligne est superieure
    a 600
31  matrix_prod_1[matrix_prod_1 > 700] #Extraction des
    coeffcients de matrix_prod_1 superieurs a 700, le
    resultat est un vecteur
32
33  #Concatenation de matrices.
34
35  rbind(matrix_1,matrix_2) #Concatenation verticale des
    matrices matrix_1 et matrix_2
36  cbind(matrix_1,matrix_2) #Concatenation horizontale des
    matrices matrix_1 et matrix_2
```

Listing 1 – Structures de données dans R

Rmarque : La fonction `apply?` : La fonction `apply` permet d'appliquer une fonction qui fonctionne sur les vecteurs aux colonnes ou aux lignes d'une matrice. C'est très utile pour éviter de faire des boucles.

Saisir le script suivant dans l'environnement de RStudio :

```
1
2  apply(matrix_1, 2, sum) #Calcul de la somme des colonnes
     de matrix_1.
3  apply(matrix_1, 1, sum) #Calcul de la somme des lignes
   de matrix_1.
4  apply(matrix_1, 1, max) #Ici on calcule le max par ligne
   .
```

Listing 2 – Structures de données dans R

III | Partie pratique

Exercice .1

1. Créer les vecteurs `vect_x = (2; 2; 2; 2; 2)` à l'aide de la fonction `rep` et `vect_y = (-1; 0; 1; 2; 3)` à l'aide de la fonction `seq`.
2. Créer une matrice `matrix_A` de `5`-lignes et `2`-colonnes dont la première colonne est formée des éléments de `vect\_x` et la deuxième des éléments de `vect_y`.
3. Modifier la deuxième ligne de `matrix_A` en `(2, 4)`.

Exercice .2

1. Créer la matrice

$$\text{matrix_A} = \begin{pmatrix} 2 & 23 & 8 \\ 10 & 6 & 90 \\ 4 & 7 & 12 \end{pmatrix}$$

   à l'aide des fonctions `cbind` puis `matrix`.
2. Calculer la moyenne et le produit des lignes et des colonnes de `M` à l'aide des commandes `apply`, `sum` et `prod`.
3. Créer une matrice `matrix_B` qui contient les deux premières ligne de `matrix_A`.
4. Calculer la somme des deux premières lignes de `matrix_A` puis la somme de la première et de la troisième colonne.
5. Que permettent de calculer les commandes `t(matrix_A)`, `det(matrix_A)` et `diag(A)` ? En déduire la trace de `matrix_A`.

LCS - ISI Mahdia - AU

6. Que permettent de calculer les commandes `matrix_A^2`, `matrix_A * matrix_A` et `matrix_A% * %matrix_A` ?
7. Que permettent de calculer les commandes `1/matrix_A`, `matrix_A^(-1)` et `solve(matrix_A)` ?
8. Résoudre le système ci-dessous :

$$\begin{cases} 2x + 23y + 8z = 5 \\ 10x + 6y + 90z = 6 \\ 4x + 7y + 12z = 7 \end{cases}$$

9. Extraire les lignes de la matrice `t(matrix_A)` pour lesquelles la somme des coeffcients est supérieure à `30` (si possible avec une seule ligne de commande).

IV | Les tableaux de données

Un tableau de données (`data.frame`) est un tableau dont les colonnes peuvent être hétérogènes. Par exemple, un `data.frame` peut contenir des colonnes de caractères et des colonnes de numériques, mais chacune des colonnes contiendra des éléments de même nature.

Il s'agit sans doute de la classe d'objet la plus importante puisqu'elle est consacrée spécifiquement au stockage des données. Chaque **colonne** décrit une **variable** alors que les **lignes** correspondent aux **individus**.

On peut alors mettre dans le `data.frame` des variables qualitatives décrites par des caractères (par exemple une colonne qui décrit le sexe `M/F` d'un individu ou sa catégorie socio-professionnelle) et des variables quantitatives décrites par des nombres (par exemple le montant des sinistres ou l'âge). Nous verrons ultérieurement que de nombreuses méthodes statistiques sont disponibles dans `R?` pour cette classe d'objet.

Pour créer un tableau de données, on peut regrouper des variables de même longueur à l'aide de la commande `data.frame(nom1 = var1, nom2 = var2, ...)`. Il est également possible de transformer une matrice en tableau de données en utilisant la commande `as.data.frame`. Enfin, c'est ce type d'objet qui est généralement créé par textbf\R lorsque l'on importe des données.

Saisir le script suivant dans l'environnement de RStudio :

```r

unif_vect <- runif(30, min = 0, max = 1) #30 realisations independantes d'une loi uniforme sur [0,1]

vector_1  <- LETTERS[1:5]
vector_2  <- LETTERS[6:10]
vector_3 <- 1:5
vector_4 <- 6:10

df <- data.frame(Age = vector_1, Prenom = vector_2, Taille = vector_3, Poids = vector_4)
#Creation d'un data.frame avec 4 variables et 5 individus.

df   # affichage

str(df) #Donne la structure de l'objet
df$Prenom
df[,1]       #On peut acceder aux variables avec $ ou []
df$Prenom[3] #Permet d'acceder au troisieme element du vecteur
df$Prenom
df[3,1]     #Donne le meme resultat
summary(df) #Resume statistique du jeu de donnees
df <- data.frame(vector_1, vector_2, as.factor(vector_3), vector_4)

#Constitution du tableau de donnes df avec la variable vector_3 declaree comme une variable qualitative (R voit alors
#les nombres comme des chaines de caracteres)

str(df)     #Remarquer que v3 est qualitative
```

(C) Dr. ABBASSI A. Department of Physics

```r
summary(df) #Remarquer que le resultat est different pour les variables qualitatives et quantitatives
matrix_1 <- matrix(1:15, nrow = 3); matrix_1 # affichage
matrix_1 <- as.data.frame(matrix_1) #Transformation de ma en un objet de type data.frame
matrix_1   # affichage
str(matrix_1)
names(matrix_1) <- c('VA','VB','VC','VD','VE')   #Donne des noms aux variables
row.names(matrix_1) <- c('I1','I2','I3')         #Donne des noms aux individus

# Manipulation des variables qualitatives

Sexe <- sample(c('H','F'),100,rep = T) #Simulation du sexe (H ou F) (variable qualitative)

#Simulation de la taille des individus selon une loi normale de moyenne 170 et ecart-type 10

Taille <- rnorm(100, mean = 170, sd = 10)
Taille[which(Sexe == 'H')] <- Taille[which(Sexe == 'H')] + 10

#On rajoute 10 pour la taille des hommes

tapply(Taille, Sexe, mean) #Calcul de la taille moyenne par sexe
tapply(Taille, Sexe, sd) #Calcul de l'ecart-type par sexe

# Il existe des jeux de donnees disponibles dans R :

data()        # Ouvre une fenetre texte listant l' ensemble des tableaux de donnees disponibles dans R
women         # Affiche le tableau de donnees women
? women       # Descriptif du tableau de donnees women
names(women) # Affiche le nom des variables de women
attributes(women) #Donne les caracteristiques du tableau de donnees women
```

```r

women$height
height
height

apply(women, 1, sum) #La fonction apply peut etre utilisee avec des data.frame
apply(women, 2, max)
```

Listing 3 – Tableaux de données dans R

V | Partie pratique

Exercice .1

On considère dans cet exercice le jeu de données `iris` disponible sous R.

1. vérifier que c'est un objet de type `data.frame`.
2. Que contient ce jeu de données ?
3. Taper la commande `summary(iris)`. Combien d'individus de l'espèce `versicolor` sont dans la base de données ? Quelle est la moyenne de la variable `Sepal.Width` ? La plus petite valeur ?
4. Créer un objet de type `data.frame`, nommé `iris2`, qui contient seulement les individus de l'espèce `setosa`
5. Trier les individus de l'espèce `setosa` en fonction de la longueur du sépale.
6. Retrouver les moyennes des variables `Sepal.Length`, `Sepal.Width`, `Petal.Length` et `Petal.Width` à l'aide des fonctions `apply` et `mean`.
7. Calculer les moyennes de la variable `Sepal.Length` pour chacune des trois espèces à l'aide des fonctions `tapply` et `mean`.

Exercice .2

lcs - ISI Mahdia | UM

Le tableau suivant détaille le nombre d'étudiants des universités de cinq académies inscrits en 2006 dans les cinq filières suivantes : `Lettres`, `Sciences`, `Médecine`, `Sport` et `I.U.T`.

Ville	Lettres	Sciences	Médecine	Sport	I.U.T.
Bordeaux	12220	6596	7223	357	2239
Lyon	15310	6999	10921	395	3111
Paris	112958	40244	46146	1247	7629
Rennes	8960	6170	4661	279	4013
Toulouse	12125	8233	6653	553	3178

Tableau 1 – Répartition des étudiants par ville et domaine d'études.

1. Créer un objet `z` de type `data.frame\verb` qui contient ce jeu de données. On renseignera en particulier le nom des colonnes (avec la commande `names`) et des lignes (avec la commande `row.names`).
2. alculer le nombre total d'étudiants dans chaque académie puis ordonner le tableau de données par ordre croissant de cette variable.
3. Calculer le nombre total d'étudiants dans chaque filière puis ordonner le tableau de données par ordre croissant de cette variable.
4. Créer un `data.frame` qui contient seulement les académies pour lesquelles l'effectif en sciences est supérieur à l'effectif en médecine (une seule ligne de commande).

VI | Manipulation des listes dans R

Une liste est une collection ordonnée d'objets, non nécessairement de même mode. Les éléments des listes peuvent donc être n'importe quel objet défini dans `R?`. On peut par exemple créer une liste qui contient un vecteur numérique et une matrice de caractères **!** Cette propriété est notamment utilisée par certaines fonctions pour renvoyer des résultats complexes sous forme d'un seul objet. La constitution d'une liste passe par la fonction `list(nom1 = el1,nom2 = el2,...)`. On peut accéder à chaque élément de la liste à l'aide de son index entre double crochets `[[...]]`, ou par son nom précédé du signe `$`.

```
1
2
3  my_list <- list(num = 1:5,y = "couleur",a = TRUE) #
       Creation d'une liste avec 3 objets
4  my_list
5  my_list$num
6  my_list$a
7  my_list[[1]] #On peut acceder aux elements d'une liste
       avec $ ou avec [[]]
8  my_list[[3]]
9
10 matrix_A = matrix(c(6,2,0,2,6,0,0,0,36),nrow = 3)
11 eigen(matrix_A) #Calcul des elements propres de la
       matrice
12 result <- eigen(matrix_A) #Le resultat est donne dans
       une liste
13 attributes(result) #Donne le nom des elements de la
       liste
14 str(result) #
15 result$values   #Valeurs propres
16 result$vectors  #Vecteurs propres
17 diag(result$values)
18 result$vectors% * %diag(result$values)% * %t(result$
       vectors)
19
20 vector_x <- list(a = 1:10,beta = exp(-3:3),logic = c(
       TRUE, FALSE, FALSE, TRUE))
21 lapply(vector_x, mean) #La commande lapply permet d'
       appliquer une fonction a chaque element d'une liste
```

Listing 4 – Listes dans R

(C) Dr. ABBASSI A. Department of Physics -

VII | Partie pratique

Exercice .1

Créer une liste qui contient votre nom de famille, le prénom de vos parents et votre département de naissance.

Exercice .2

On reprend la matrice `matrix_A` de l'exercice `2p.2`.

1. Calculer les valeurs propres et les vecteurs propres de `matrix_A`. La matrice est-elle diagonalisable ?
2. Déduire de la question précédente le calcul de `matrix_A^n` pour `n = 10` et `n = 50` (sans utiliser une boucle `!`).

VIII | Quelques éléments de programmation

Nous traitons succinctement dans cette section des instructions de condition et de répétition. Il s'agit des commandes `if`, `while`, `for` et `repeat`. Une boucle `repeat` est toujours exécutée au moins une fois. Elle devra comporter un test d'arrêt qui utilisera la commande `break`. La commande `break` peut aussi être utilisée pour sortir d'une boucle `for` ou `while`. La commande `next` permet le passage immédiat à la prochaine itération d'une boucle `for`, `while` ou `repeat`.

Le bloc d'instruction est délimité par des accolades (certains langages utilisent les commandes `begin` et `end`). La condition est délimitée par des parenthèses.

```
1
2  # for Loop :
3
4  x <- NULL #Initialisation
5
6  for (i in 1:10){ #i va varier de 1 a 10
7   x[i] <- i #Instruction executee pour i = 1 puis i =
       2...puis i = 10
8  } #Indique la fin de la boucle for
9  x
10 s <- 0
11 x <- rnorm(10000)
12
13 for (i in 1:10000){
14  if (x[i] < 10){
15   s <- s + x[i]
16  } else{
17     s <- s + 2 * x[i]
18   }
19 }
20 s
21 sum(x[x < 10]) + 2 * sum(x[x >= 10]) #Permet d'eviter la
       boucle for : plus efficace !
```

Listing 5 – Structures de contôle du flux dans R

Remarque :

Lorsqu'on commence un bloc d'instruction avec le signe `{`, le prompt est modifié dans la console et le signe `+` apparaît. Afin de forcer `R?` à sortir du bloc d'instruction, on peut appuyer sur la touche `echap`.

```
1
2  # while Loop :
3
4  x <- NULL #Initialisation
5
6  i <- 0 #Initialisation
7  while(i < 10){ #Condition d'arret : R execute le bloc d'
       instruction tant que i est inferieur a 10
8
9   i <- i + 1 #Sans cette instruction la boucle while est
       sans fin
10  x[i] <- i
11 } #Fin du while
12 x
13
14 # repeat Loop :
15
16 x <- NULL
17 i <- 1
```

(C) Dr. ABBASSI A. Département de Physique - ISI Mahdia

```r

 repeat{
  x[i] <- i
  if (i >= 10){
  break #On sort de la boucle repeat lorsque i depasse 10
 } #Fin du if
 i <- i+1
 } #Fin du repeat
```

Listing 6 – Structures de contôle du flux dans R

Remarque :

Il faut éviter d'utiliser des boucles avec `R?` puisqu'elles sont généralement inefficaces (**temps de calcul importants**). Il est souvent possible de vectoriser les calculs pour éviter les boucles explicites (les opérateurs matriciel de `R?` utilisent des boucles `C?` beaucoup plus rapides), ou encore d'utiliser les fonctions `outer`, `apply`, `lapply`, `tapply` et `mapply` pour réaliser les boucles de manière plus efficace.

```r

x <- matrix(rep(1, 10^8),nrow = 10^4) #Creation d'une
    grande matrice

a <- Sys.time() #Utilise pour calculer les temps de
    calcul

S <- rep(0, nrow(x)) #Calcul de la somme des lignes avec
    des boucles for imbriquees

for (i in 1:nrow(x)){
 tmp <- 0
 for (j in 1:ncol(x)){
  tmp <- tmp + x[i,j]
}
S[i] <- tmp
}

b <- Sys.time()
S2 <- apply(x,1,sum) #Calcul de la somme des lignes avec
    apply
c <- Sys.time()
S3 <- x %% rep(1,nrow(x)) #Calcul de la somme des lignes
    avec un produit matriciel
d <- Sys.time()
c(b - a, c - b, d - c) #Temps ecoule pour chaque methode
    : le produit matriciel est beaucoup plus rapide !
```

Listing 7 – Temps de calcul des boucles dans R

Remarque 1 N'hésiter pas à visiter le **site officiel de R Project**, et découvrir la documentation disponible : https ://www.r-project.org/.